AVR ASSEMBLER · ATMEGA8

Два расписания без очереди сообщений

Программа периодически передаёт **ping\r\n** и **pong\r\n** через USART. Периоды и тексты можно менять командами. Timer1 непрерывно считает такты процессора и обслуживает два независимых расписания через OCR1A и OCR1B.

Что здесь измеряется

Точность определена как отклонение **записи первого байта строки в регистр UDR** от назначенного срока D: **0 ≤ t − D ≤ 4 такта CPU**. Это программное начало передачи, не промежуток между словами и не измерение фронта на выводе TX.

Слайд не уточняет точку измерения. Здесь выбрана и проверена точка OUT UDR. Она не выдаётся за согласованную с преподавателем трактовку. Если требуется точность физического стартового бита TX, приложенные проверки этого не подтверждают.

Почему строки больше не сталкиваются

Первое расписание запускается от общего момента отсчёта, второе — позже на PHASE. Периоды остаются независимыми. Перед принятием настроек программа проверяет, что этот сдвиг позволяет передавать обе строки целиком и оставляет время на обслуживание команд.

Обработчик входит за 512 тактов до срока, заранее получает первый символ и ждёт аппаратный флаг сравнения. Последнее ожидание занимает четыре такта на оборот. Сразу после обнаружения флага выполняется OUT UDR. Следующий срок вычисляется от предыдущего: **D_next = D + T**.

Начальные параметры

CPU — реальные 8 МГц; USART — 500000 бод, 8N1. Периоды — 16000 и 24000 тактов (2 и 3 мс). Для исходных строк PHASE = 3008 тактов (376 мкс). Первые сроки: T1 и PHASE + T2 после момента отсчёта.

Исходник: serial_scheduler/serial_scheduler.asm; образ: serial_scheduler.hex. Размер: 1466 байта кода и 95 байт объявленных данных, без учёта стека и промежутков адресов SRAM.

Условие взято из Achievement #1 первой лекции. Алгоритм, фазовый сдвиг, протокол и программа разработаны для этой работы. Регистры и формат передачи описаны в ATmega8 datasheet.

МАТЕМАТИЧЕСКАЯ МОДЕЛЬ

Проверка параметров и минимальная сумма

L1 и L2 — длины текста без CRLF, от 0 до 16. Один байт 8N1 занимает 160 тактов. Длительности строк: C1 = (L1 + 2) × 160 и C2 = (L2 + 2) × 160. Между строками резервируется G = 2048 тактов на ранний вход в обработчик, обслуживание и ответ на команду.

```
g = gcd(T1, T2)
PHASE = C1 + G
accept: g >= C1 + C2 + 2*G
```

Откуда берётся условие

Разности сроков двух периодических потоков имеют вид PHASE + k × g. Чтобы строки и служебные окна не пересекались, нужны два неравенства: **PHASE ≥ C1 + G** и **g − PHASE ≥ C2 + G**. При выбранном сдвиге они выполняются ровно тогда, когда g ≥ C1 + C2 + 2G.

Это точный критерий принятия для реализованной политики резервирования. Он не утверждает, что меньшие периоды невозможны при другом алгоритме, меньшем резерве или другой скорости USART.

Доказательство минимума

Оба периода — положительные кратные g. Поэтому **T1 + T2 ≥ 2g ≥ 2(C1 + C2 + 2G)**. Равенство достигается при **T1 = T2 = C1 + C2 + 2G**. Для всех поддерживаемых длин эта пара лежит в разрешённом диапазоне до 57344 тактов.

Для **ping/pong**: C1 = C2 = 960, каждый минимальный период равен **6016 тактам**, минимальная принимаемая сумма — **12032 такта**. На 8 МГц это 752 мкс на период и 1,504 мс в сумме. PHASE = 3008 тактов.

Проверка на границе: P 1780 1780 принимается; P 177F 177F отклоняется без изменения настроек. Для пустых строк, состоящих только из CRLF, минимум равен 4736 + 4736 = 9472 такта.

Счётчик продолжает работать

НОД вычисляется при разрешённых прерываниях: таймерные сообщения продолжают передаваться во время расчёта. Принятые параметры публикуются атомарно. Оба расписания назначаются заново от текущего счётчика; TCCR1B и TCNT1 при этом не записываются.

КОМАНДЫ И ПОВЕДЕНИЕ

Управление через USART

Каждая команда — одна строка ASCII, завершённая LF. Следующую команду отправляют после ответа **!OK\r\n** или **!ERR\r\n**. Периоды записываются четырьмя шестнадцатеричными цифрами в тактах CPU.

T1 5DC0 Период первого расписания: 24000 тактов.

T2 3E80 Период второго расписания: 16000 тактов.

P 1780 1780 Одновременное изменение обоих периодов на 6016 тактов.

S1 hello / S2 world Замена первого или второго текста.

S1 + пробел Пустой текст первого сообщения; CRLF сохраняется.

R Перезапуск обоих расписаний с рассчитанным фазовым сдвигом.

Команда P добавлена для перехода между допустимыми парами: поочерёдное изменение периодов могло бы создать недопустимую промежуточную пару. Команды T1 и T2 остаются доступны.

Проверки перед изменением

Проверяются синтаксис, диапазон периодов 1..57344, длины строк и совместимость расписаний. Текст содержит до 16 печатных ASCII-символов и не начинается с ! — это префикс служебных ответов. CRLF добавляется автоматически. Неверная команда возвращает !ERR и сохраняет прежние настройки.

Фрагмент, определяющий точность

```
wait_a:
    in   r16,TIFR
    sbrs r16,OCF1A
    rjmp wait_a
    out  UDR,r20
```

Пока флаг не установлен: IN — 1 такт, SBRS — 1, RJMP — 2; итого 4. Регистры и первый символ подготовлены заранее, другие прерывания не вмешиваются в финальное ожидание. Момент сравнения смещён на три такта раньше D. Фактическое отклонение проверяет отдельный стенд по исполняемому HEX.

Приём байтов опрашивается также во время раннего ожидания и передачи остатка строки. Запрещена отправка нескольких команд подряд без ожидания ответа.

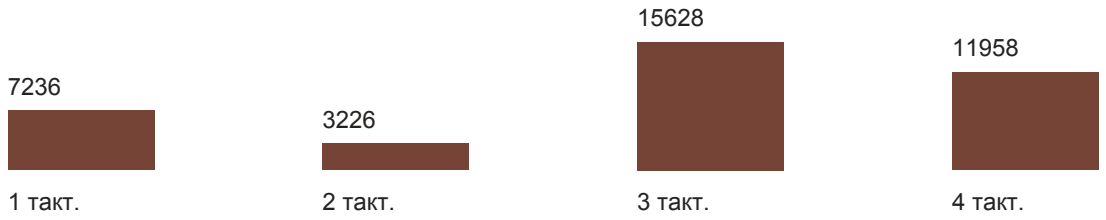
ВОСПРОИЗВЕДЕНИЕ И ГРАНИЦЫ ИЗМЕРЕНИЯ

Результат проверки

AVRA собрал программу без ошибок. Выполнено **337 сценариев**, измерено **38048 начал строк**. Во всех измерениях отклонение первого OUT UDR от срока составило **1–4 такта**. Остановок счётчика и пропусков в проверенных установившихся расписаниях не обнаружено.

Проверены все 289 сочетаний длин 0..16 на границе допуска, исходные и минимальные периоды, переполнения 16-битного счётчика, перезапуск, ошибки команд, сохранение параметров при отказе, длинный расчёт НОД и разные фазы приёма байтов на скорости 500 кбод.

Распределение измеренных отклонений



Как повторить

```
cd serial_scheduler
python3 build_and_check.py
```

Нужны Python 3, git, make и C-компилятор. Скрипт получает зафиксированные версии AVRA/simavr, собирает HEX и запускает проверки. Сохранённый результат: checks/result.json; новый: build/command-tests/result.json.

Что симуляция не доказывает

Тестовый стенд наблюдает запись UDR и счётчик Timer1, а не физический TX. Для модели UART simavr явно установлена длительность кадра 160 тактов: исходная модель некорректно учитывает общий адрес UBRRH/UCSRC и бит чётности ATmega8. Выполнение инструкций, Timer1 и временные отметки не изменяются. На плате программа не испытывалась.

Источник аппаратных сведений: ATmega8(L) Complete Datasheet, разделы Timer/Counter1 и USART. Показатели тестов получены из приложенного прогона; это не результаты измерения осциллографом.